COINjecture Network

A Mathematics-Backed Peer-to-Peer Network

COINjecture.com

Whitepaper
Version 2.6, May 30, 2026

Authors: Sarah Marin (Founder), Al Zalewski

Abstract: COINjecture is a purely peer-to-peer network that harnesses the solve-verify asymmetry of NP problems to replace the traditional proof-of-work hashing mechanism (e.g., SHA256) needed to outpace attackers and secure a robust network of digital payments. Chain validity derives from solution correctness and cumulative mathematical work, ensuring robust consensus without centralized validators. Difficulty is calibrated to instance hardness, not just solution validity, ensuring trivial solutions yield negligible rewards. The record formed in the process of solving and verifying these problems in the complexity class NP creates a computational marketplace where miners earn rewards for solving computationally hard problems while enabling secure, decentralized transactions.

Code Base:
Active: https://github.com/COINjecture-Network/COINjecture2.0
Archive: https://github.com/coinjecture-network/coinjecture
Archive: https://gitlab.com/beanapologist/coinjecture
Archive: https://codeberg.org/beanapologist/COINjecture

Introduction

Traditional proof-of-work blockchain systems, like Bitcoin, rely on the Secure Hash Algorithm 256-bit (SHA-256) as the mechanism to verify transactions, link blocks, and maintain network integrity and chain immutability. The SHA-256 algorithm is applied to hash a block header repeatedly until one is found with a specific prefix of leading zeros. [1] During this process a random number called a nonce is changed over and over with each attempt in a brute-force effort to find the solution. This hash calculation provides no scientific or mathematical value outside of network security and the computational resources required to perform this brute-force search resulted in the consumption of over 150 TWh in 2024, according to the Cambridge Bitcoin Electricity Consumption Index. [2] This computation provides network security but no additional utility.

What is needed is a cryptographically secure electronic payment system that generates utility beyond security. Building on Bitcoin's foundation of decentralized consensus through proof-of-work, COINjecture proposes to replace brute-force hash search with NP problem solving as the core proof-of-work mechanism that records verifiable, immutable computational data on-chain. Consensus is derived from the mathematical correctness of submitted blocks and the overall work performed by the chain. The computational work that fuels chain growth and provides network consensus also allows for the creation of a computational marketplace in which miners are rewarded for solving computationally hard problems submitted by users or generated by the network state.

This Proof of Useful Work (PoUW) model is theoretically grounded and has prior art in deployed chains [3, 4]. It has recently been extended to include NP problem optimization specifically [5]. This peer-to-peer network directly rewards participants proportionally based on their mathematical contributions with no fixed fee schedules or required transaction fees. Where competitors utilize consensus tied to a fixed problem, COINjecture is designed to be extensible across a registry of problem types and current testnet operations utilize subset sum, traveling salesman problem (TSP), and Boolean Satisfiability Problems (SAT). As participation and demand grow, additional NP problems will be added to the registry. There are theoretically thousands of problems spanning areas including logistics, cryptography, scheduling and optimization, and biology, and hundreds that have been mathematically proven to be part of the NP family.

Transactions

COINjecture follows the standard UTXO model in which we define an electronic coin as a chain of digital signatures. This transaction format follows Bitcoin's UTXO model without modification beyond the addition of a required proof bundle field in the block header. Each owner transfers the coin by digitally signing a hash of the previous transaction and the public key of the next owner. In addition, all blocks in our network must contain not just transactions, but also proofs of computational work (e.g., NP problem solutions) that secure consensus and provide economic utility. This data alongside transactions is broadcast to all nodes in the peer network.

Timestamp Server

Like Bitcoin, we employ a timestamp server that proves that the data must have existed at the time of announcement. Each timestamp includes the previous timestamp in its hash as well as the computational proofs (problem + solution) of the block, forming a chain. The proof bundle includes both a commitment timestamp (T_1) and reveal timestamp (T_2) determined by the salt-commit-mine-reveal protocol outlined in the subsequent section.

Proof-of-Work

To implement a distributed timestamp server on a peer-to-peer basis, we use a proof-of-work system based on NP problems rather than hash collision. The defining feature of NP problems is that a candidate solution can be verified as correct or incorrect, but not necessarily solved, in polynomial time—hence the name *nondeterministic polynomial time*—so that given a potential answer to the problem, you can check if it is correct quickly, even if finding the answer in the first place is difficult or time-consuming. For instance, a sudoku puzzle may take several minutes to solve, but can be checked for correctness quickly by ensuring no row or column contains two occurrences of the same number.

a. Commitment Protocol

To prevent grinding or generating many problems to find easy instances, we employ a succinct salt-commit-mine-reveal protocol:

1. Miner commits to the problem: $commitment = H(problem\_params \,\|\, salt)$
2. Miner solves problem
3. Miner computes: $reveal = H(problem\_params \,\|\, salt \,\|\, H(solution))$
4. Miner publishes solution bundle
5. If validated, new epoch salt deterministically derived from accepted block hash

The epoch salt is deterministically derived from the parent block hash to prevent pre-mining activities. Commitment cryptographically binds to solution via $H(solution)$ while the hash reveals nothing about the actual solution structure. As the system encourages multiple miners to compete on the same problem instance, any miner who delays their commitment or artificially inflates their solve time risks being outpaced by a competitor. This competitive pressure enforces honest timing without requiring a trusted clock. Even if solve time is inflated, reward is bounded proportionally by the emission formula (see *Fee Schedules*).

b. Problem Types

The commit-reveal protocol is designed to be extensible and allows the network to use different NP-complete, co-NP-complete, and NP-hard problem types (any problem whose solution admits a short, polynomial time-checkable certificate). Each problem type is included in a registry based on their unique computational characteristics. These characteristics are then used to calculate the difficulty and work score of the solution set. See Appendix A for a complete problem registry and list of proposed future additions.

c. Work Score Calculation

Unlike traditional hash-based proof-of-work, where `work` is primarily the number of hashes attempted (inferred from the difficulty target), our work score is a measurement of computational complexity that takes into account solve and verification time, the correctness, and optimality of the solution. This work score is not arbitrary and is constrained by network competition and verification of solutions.

The raw work score is anchored to the time asymmetry between solve time and verify time—the fundamental characteristic that all NP problems share:

$$raw\_work = \log_2\left(\frac{solve\_time}{verify\_time}\right).$$

The *solve_time* is measured as the block-timestamp difference between commitment inclusion and solution reveal, anchored to the timestamp server's consensus chain. *Verify_time* is the network-measured, instance-fixed denominator. Thus, the asymmetry is anchored to a real property of the solution, not just wall-clock.

Using $\log_2$, we can measure computational difficulty in a unit comparable across problem types without knowing anything about problem structure (*problem_params*). The complete work score is time asymmetry with quality corrections as multiplicative adjustments:

$$work\_score = \log_2\left(\frac{solve\_time}{verify\_time}\right) \times quality\_score,$$

where $quality\_score \in [0, 1]$ (exact optimal solution = 1, valid but suboptimal < 1). Solution quality is established for each problem type in the problem registry and scored against the best solution found in the current epoch. Both time asymmetry and solution quality are verifiable by the network and cannot be gamed via self reporting.

d. Difficulty Adjustment

The difficulty weight affects what happens *before* mining (problem generation). The work score measures what happened *after* mining (solution quality). The difficulty target adjusts based on the fixed window of $N = 20$ blocks to allow for stable analysis and to help support our optimal block time of 10 seconds. The adjuster requires at least $N/2 = 10$ blocks of history before making its first adjustment, and defers adjustment entirely when variance within the window is high ($\sigma > 0.8\mu$). This value represents a tradeoff: larger N produces smoother adjustment but slower response to hashpower changes; smaller N responds faster but risks oscillation.

At a 10-second block time, $N = 20$ corresponds to approximately 200 seconds of network history—long enough for statistical stability, short enough to adapt within minutes. This difficulty adjustment allows work scores to maintain stable block time and for decentralized participation in the mining pool. The difficulty adjuster also prevents sandbagging by calibrating expectations to recent network performance, so artificially inflated solve times produce scores that deviate from the target without conferring a competitive advantage.

Network

The steps to run the network are as follows:

1. New transactions are broadcast to all nodes
2. Miners select problems from epoch salts (user-submitted or consensus-generated)
3. Miner binds to problem and creates commitment
4. Miners solve problems
5. When a valid solution and header are found, broadcast to all nodes
6. All nodes validate the solution and accept the block if valid
7. Nodes express acceptance by working on the next block using the salted hash of the accepted block

Nodes always consider the chain with the highest cumulative work to be correct and work on extending it. The cumulative work score is also the tiebreaker mechanism in the event that two nodes broadcast different versions of the next block simultaneously. The asymmetry of NP problems (fast verification, slow solving) means that nodes can quickly verify competing chains during a fork to determine which one is valid and has the most "weight" according to the tiebreaker rule.

Problem Pool

The network maintains two problem sources: consensus problems and user problems. Consensus problems are generated deterministically from network state for base rewards. User problems are submitted with escrowed bounties. User problem work scores contribute to chain weight identically to consensus work. Miners may allocate their compute resources freely between consensus problems (block rewards) and marketplace problems (bounties).

Incentive

By convention, the first transaction in a block is a special transaction that starts a new coin owned by the creator of the block. This adds an incentive for nodes to support the network and provides a way to initially distribute coins into circulation, since there is no central authority to issue them.

Unlike Bitcoin, which uses the analogy of gold miners expending resources to add gold into circulation, COINjecture incentive is derived through the measured performance of the nodes completing the work, like a promotion based on the work you performed. In our case, it is empirically measured against the actual performance of the chain and includes problem solution quality, solution speed, and the time it takes to verify correctness. This quantifies "good work" and rewards it proportionally using the emission formula:

$$mint = \left\lfloor \frac{w_{\text{trunc}} \cdot S \cdot K}{W_{\text{parent}}} \right\rfloor,$$

where w_{trunc} is the truncated work score of the current block, K is the emission multiplier (set at $K = 50$ and chosen to produce approximately 50 coins on the first block after genesis), S is the fixed-point scaling factor (10^{12} atoms/coin), and W_{parent} is the parent cumulative work. This mint issuance is separate from fork-choice weight (cumulative w_{trunc}) as reward does not affect security weight.

Each block's reward depends only on its own work and the parent's cumulative work score, with no future dependence. The monotonicity created by proportional rewards based on work (more work yields more reward) is proven in Lean [6]. By design, you cannot inflate solve time without losing blocks to faster miners. This tension between work score incentive and racing incentive (high time = higher score vs. lower time = block winner) further helps ensure honest participation in the system.

Fee Schedules

COINjecture does not utilise predetermined fee schedules. Instead, incentive emerges from the dynamics of the decentralised network state itself. Consensus transactions carry no fees beyond an optional priority tip because miners are already compensated by block rewards. User-submitted problems require escrowed bounties, which serve as both payment for computational work and natural spam deterrence. User-submitted problems also require a network fee upon submission. There are no "wasted" transactions because every submission either contributes to consensus or funds useful computation.

The emission formula creates nonzero asymptotic growth—as W grows, per-block issuance falls. No hard caps or limits, only the pressure of consensus reward decay. As the network matures, fee incentives will shift from consensus problems as the primary driver to user-submitted bounties; however, rewards for consensus transactions will always be compensated. As each block's mint is proportional to $w_{\text{trunc}}/W_{\text{parent}}$, even a successfully inflated work score would yield diminishing marginal reward and an attacker optimising for score inflation gains an ever-smaller share of a growing total.

These principles ensure the system adapts naturally to its own evolution without manual intervention and that rewards scale dynamically. The reward structure may also help encourage nodes to stay honest and drive difficulty higher. If a greedy attacker is able to recalculate all of the correct solutions and produce the same quality of work as honest nodes, they would have to choose between using it to defraud people by stealing back their payments, or using it to generate new coins. They ought to find it more profitable to play by the rules—rules that favour them with more new coins than everyone else combined—than to undermine the system and the validity of their own wealth.

Thus, the steady addition of useful computational work to the chain serves as both a security mechanism and provides economic utility. Nodes are incentivised to perform and verify real mathematical work quickly, creating value beyond network security.

The Conjecture

The preceding sections describe a designed network of competing miners whose collective behaviour is shaped by the parameters established (e.g., solve times, difficulty adjustments,

block production). The conjecture is not that these quantities are fixed constants at genesis. It is that a network built from these axioms and that measures and distributes rewards accordingly will empirically converge toward a predicted equilibrium; both the COIN and conjecture in COINjecture. That prediction is falsifiable by running the network and comparing measurements to the formulas in the Calculations section.

a. Design Axioms

We establish two core design constraints or axioms for our system dynamics via symmetry and unit normalisation. COINjecture tests whether a decentralised network built on these axioms naturally converges to a mathematically predicted steady coherence state.

Let $\mu = x+iy$ represent the consensus eigenvalue where the unit-modulus constraint $|\mu|^2 = 1$ represents unitarity conservation.

1. Unit normalisation: $x^2 + y^2 = 1$
2. Symmetry condition: $|x| = |y|$

From symmetry, $x^2 = y^2$. Substituting into the unit constraint: $2x^2 = 1$, giving $|x| = |y| = 1/\sqrt{2}$. Numeric checks of these axioms are verified at sample points in `DesignAxioms.lean` [7]. The constant $1/\sqrt{2}$ enters the protocol as the difficulty target multiplier η, while the eigenvalue's phase is interpretive and does not feed difficulty, reward, or fork choice. The two axioms determine $|x| = |y| = 1/\sqrt{2}$ but leave four candidate points on the unit circle. The whitepaper selects $\mu = (-1 + i)/\sqrt{2}$ from the additional constraint Re < 0, formalised as `mu_unique` in `ComplexDecomposition.lean` [8]. The modulus/argument decomposition used in the next subsection is the same file's anchor/coherence factorisation.

b. Falsifiability

The conjecture predicts the convergence $r = 1$ with transient deviations decaying as $C(r^n) = \text{sech}(n \cdot \log r)$, where the coherence function $C(r) = 2r/(1+r^2)$ attains its maximum $C(1) = 1$ at equilibrium. For our network, r is the dimensionless ratio between actual block solve time (the modulus) and the difficulty adjuster's optimal target (the argument). This creates natural epistemic asymmetry: the numerator, block solve time, is verified and fixed by the consensus chain once a block is accepted; the denominator, the difficulty adjuster's optimal target, is self-referential and is recomputed from the network's own median. Sech is the natural envelope for systems with reflective symmetry about equilibrium, where perturbations in either direction are penalised identically—$C(r) = C(1/r)$. Each component plays an important role: what the chain knows vs. what the chain expects. The conjecture is that the network's measurement-and-reward dynamics drive this ratio toward 1.

The conjecture yields three testable predictions:

1. Perturbation recovery should follow the sech profile rather than exponential decay.
2. The difficulty oscillation should exhibit a characteristic periodicity as the system converges to the lowest overhead state.
3. The work score distribution should be symmetric about $\log(r) = 0$, reflecting $C(r) = C(1/r)$.

Failure of any of these constitutes falsification. By building a network that measures itself and distributes rewards accordingly (e.g., block solve times, difficulty scaling, optimality), we can test whether the predicted coupling between resolved work and self-referential target sustains stable operation.

Reclaiming Disk Space

Once the latest block in a chain is buried under enough blocks, solutions to problems can be discarded by mining nodes to save disk space. To facilitate this without breaking the block's hash, solutions and transactions are hashed in a Merkle tree. Block headers form the Merkle tree, with only the root included in the block's hash. Old blocks can be compacted by stubbing off branches of the tree.

A block header with no transactions or solutions is about 1 KB. If we suppose that blocks are generated at our optimal block time of 10 seconds—long enough for NP solving to be meaningful, short enough for reasonable transaction finality, and enough overhead margin for verification plus propagation—header storage is approximately 3.2 GB per year. With commodity servers commonly shipping with 1 TB or more of storage as of 2025, this should not be a constraint even over decades of operation.

The network supports different node types with varying storage requirements:

- **Light Nodes:** Store only block headers and verify commitments. Can request full solutions from full nodes when needed. Suitable for mobile mining and everyday transactions.
- **Full Nodes:** Store complete blockchain and recent proof bundles (e.g., last N epochs). Validate all transactions and solutions. Can prune old solutions after sufficient confirmations.
- **Archive Nodes:** Maintain complete historical record of all proofs. Enable academic verification, reproducibility, and serve as reference for the network.

Mining can occur on any node type. Light miners request work from full nodes, solve problems locally, and submit solutions. This enables mining on resource-constrained devices while maintaining network security through full node validation. For user-submitted problems, solutions are recorded on-chain but full proof bundles can be pruned by non-archive nodes after the user retrieves their solution.

Privacy

Traditional privacy models rely on limiting access to information. COINjecture requires transparency of measurements while preserving user privacy of methods.

a. Public Information (Required for Verification)

The network must verify computational work, which requires:

- Problem parameters and solution
- Commitment timestamp (T_1) and reveal timestamp (T_2)

- Work score measurements (*solve_time*, *verify_time*, quality)

All nodes independently verify solution correctness and quality. This transparency is necessary for trustless consensus.

b. Proprietary Information

Computational methods remain proprietary while measurements remain verifiable. This preserves both marketplace trust (public verification) and competitive advantage (private innovation). Transaction privacy follows Bitcoin's model using pseudonymous addresses. The epoch salt, deterministically derived from the parent block hash, prevents block linking to a common owner and ensures problems cannot be pre-computed before commitment. This ensures that the network learns **WHAT** you solved (problem and solution), **WHEN** you solved it (T_1 and T_2), and **HOW LONG** it took ($solve\_time = T_2 - T_1$). The network never learns **HOW** you solved it (algorithm) or **WHY** your implementation is efficient (proprietary optimisations).

Calculations

Unlike Bitcoin, which assumes each block adds the same amount of weight to the chain, each new block in our chain has a different truncated work score. Consider an attacker controlling fraction q of total work-score production rate, with honest miners controlling $p = 1 - q$. Honest nodes verify before accepting (rejecting any block which fails the deterministic checker) and follow the heaviest valid tip—the chain with greatest cumulative truncated work $W = \sum w_{\text{trunc}}$ from the common ancestor. To replace the honest tip, an attacker must produce a valid alternate chain whose cumulative W exceeds the honest chain's.

The asymmetry of NP problems (exponential solve, polynomial verify) means nodes can quickly verify competing chains during forks to determine cumulative work W. However, one challenge is quantifying how much work an attacker must out-produce. The work score calculation utilises $\log_2$ to establish a bit equivalent. Thus, cumulative W is measured in bits and an attacker's deficit is a bit-deficit, not a block-count deficit.

Where Bitcoin treats honest and attacker block production as competing Poisson processes whose event rates are proportional to hash-power shares p and q [1], COINjecture treats them as Poisson processes whose rates are proportional to work-score production shares, with W-bits replacing block counts as the unit of "length." An attacker who starts z bit-confirmations behind must close that bit-deficit before the honest chain extends further. When $p > q$, the probability of ever doing so drops exponentially in z. The leading-order bound is an analogy of Bitcoin:

$$P(\text{attacker succeeds}) \approx \left(\frac{q}{p}\right)^z.$$

The P column is the simple bound $(q/p)^z$. Each additional honest confirmation multiplies the attacker's failure probability by roughly q/p. **Lean proves the integer inequalities, not the decimal cells.** For positive integers with $q < p$ and $z > 0$, Lean proves $q^z < p^z$—the statement that an attacker's success probability is strictly below 1 at every confirmation depth [9]. Lean additionally proves that cumulative work strictly increases when a positive-work block is appended. An attacker cannot fake q as every block needs a verified solution,

trivial problems score ≈ 0, and padding volume with self-funded bounties costs fees. Therefore, out-producing honest work is an adversary's only path to rewriting history. That path closes exponentially as confirmation depth expands.

Each inequality below is Lean-verified [9]. We note that NP solving, unlike SHA-256 hashing, is not memoryless—a miner mid-solve is closer to a solution than one who just started. In practice, this means the Poisson bound is conservative; honest miners who have invested solve time are harder to displace than the model assumes. As such, the probabilities are computed for display only and will be more tightly bound in future work.

q	z	Bound	Lean	P
10%	5	$(1/9)^5$	< 1	$\approx 1.7 \times 10^{-5}$
10%	10	$(1/9)^{10}$	< 1	$\approx 2.9 \times 10^{-10}$
30%	10	$(3/7)^{10}$	< 1	$\approx 2.1 \times 10^{-4}$
49%	5	$(49/51)^5$	< 1	$\approx 8.2 \times 10^{-1}$
49%	20	$(49/51)^{20}$	< 1	$\approx 4.5 \times 10^{-1}$

Therefore, COINjecture remains secure as long as no single solver out-produces everyone else across all work types.

Conclusion

We have proposed a blockchain system that replaces traditional SHA-256 proof-of-work mining with NP mathematical computations to create a marketplace for novel computational data. Nodes operate independently with minimal coordination, yet every node can participate in the validation process commensurate with resources. The mathematical work performed by the network serves as the consensus mechanism and allows us to empirically measure the behaviours of a dynamical system in real time. This creates the first blockchain where proof-of-work secures the network, provides economic utility, and advances complexity research. This combination of incentive (COIN) and incomplete information (conjecture) forms the basis for our peer-to-peer system.

References

References

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. https://bitcoin.org/bitcoin.pdf

[2] Cambridge Centre for Alternative Finance, "Cambridge Bitcoin Electricity Consumption Index (CBECI)." https://ccaf.io/cbnsi/cbeci

[3] M. Ball, A. Rosen, M. Sabin, and P. N. Vasudevan, "Proofs of Useful Work," *IACR Cryptology ePrint Archive*, Report 2017/203, 2017. https://eprint.iacr.org/2017/203

[4] S. King, "Primecoin: Cryptocurrency with Prime Number Proof-of-Work," 2013. https://primecoin.io/primecoin-paper.pdf

[5] C. G. Oliver, A. Ricottone, and P. Philippopoulos, "Proposal for a Fully Decentralized Blockchain and Proof-of-Work Algorithm for Solving NP-Complete Problems," arXiv:1708.09419, 2017.

[6] COINjecture Network, "Rewards.lean," COINjecture2.0 repository, 2026. https://github.com/COINjecture-Network/COINjecture2.0/blob/main/lean4/Coinjecture/Rewards.lean

[7] COINjecture Network, "DesignAxioms.lean," COINjecture2.0 repository, 2026. https://github.com/COINjecture-Network/COINjecture2.0/blob/main/lean4/Coinjecture/DesignAxioms.lean

[8] COINjecture Network, "ComplexDecomposition.lean," COINjecture2.0 repository, 2026. https://github.com/COINjecture-Network/COINjecture2.0/blob/main/lean4/Coinjecture/ComplexDecomposition.lean

[9] COINjecture Network, "SecurityCalculations.lean," COINjecture2.0 repository, 2026. https://github.com/COINjecture-Network/COINjecture2.0/blob/main/lean4/Coinjecture/SecurityCalculations.lean

[10] R. M. Karp, "Reducibility Among Combinatorial Problems," in *Complexity of Computer Computations*, Plenum Press, pp. 85–103, 1972.

[11] E. Horowitz and S. Sahni, "Computing Partitions with Applications to the Knapsack Problem," *Journal of the ACM*, 21(2):277–292, 1974.

[12] N. Howgrave-Graham and A. Joux, "New Generic Algorithms for Hard Knapsacks," in *Advances in Cryptology – EUROCRYPT 2010*, LNCS, vol. 6110, pp. 235–256, Springer, 2010.

[13] M. Held and R. M. Karp, "A Dynamic Programming Approach to Sequencing Problems," *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962.

[14] S. Lin and B. W. Kernighan, "An Effective Heuristic Algorithm for the Traveling-Salesman Problem," *Operations Research*, 21(2):498–516, 1973.

[15] S. A. Cook, "The Complexity of Theorem-Proving Procedures," in *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing*, pp. 151–158, 1971.

[16] J. P. Marques-Silva and K. A. Sakallah, "GRASP: A Search Algorithm for Propositional Satisfiability," *IEEE Transactions on Computers*, 48(5):506–521, 1999.

[17] U. Schöning, "A Probabilistic Algorithm for k-SAT and Constraint Satisfaction Problems," in *Proceedings of the 40th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pp. 410–414, 1999.

Problem Registry

Our current testnet employs Subset Sum, Traveling Salesman Problem (TSP), and Boolean SAT. Each problem type implements a `ProblemDescriptor` trait that declares its computa-

tional characteristics. The difficulty adjuster and work score calculator operate generically over this trait—adding a new problem type requires only a trait implementation and registration.

The registry's `base_difficulty_weight` informs the difficulty adjuster's cross-problem size calibration but does not enter the work score formula, which remains problem-type agnostic.

a. Subset Sum

Subset Sum is a classic NP-Complete problem with no known polynomial-time algorithm [10]. It asks if any subset of a given set of positive integers sums to a specific target value:

$$\text{Given } S = \{s_1, s_2, \ldots, s_n\} \text{ and target } T, \text{ find } S' \subseteq S \text{ such that } \sum_{s \in S'} s = T.$$

Subset Sum serves as the canonical reference type—all size ratios and difficulty weights in the registry are normalised relative to it. Verification is $O(n)$: sum the claimed subset and compare to T. Quality is binary (correct = 1.0, incorrect = 0.0). Best known algorithms include Horowitz–Sahni meet-in-the-middle $O(2^{n/2})$ and Howgrave-Graham–Joux $O(2^{0.337n})$ [11, 12].

b. Traveling Salesman Problem (TSP)

Given n cities and a distance matrix, find the shortest Hamiltonian tour visiting each city exactly once and returning to the start. NP-hard. Verification is $O(n)$: confirm the tour is a valid permutation, then sum edge weights.

TSP is the primary vehicle for testing quality-weighted work scoring because it admits a quality gradient—a valid tour that visits all cities is correct, but shorter tours score higher. Quality = $1/(tour\_length + 1)$, scored relative to the best solution in the current epoch. Best known exact algorithm is Held–Karp $O(2^n \cdot n^2)$; practical solvers use Lin–Kernighan heuristics [13, 14].

c. Boolean SAT

Given a Boolean formula φ in conjunctive normal form (CNF) with n variables and m clauses, find a satisfying assignment. NP-complete (Cook–Levin theorem) [15]. Verification is $O(n \cdot m)$: substitute the assignment and confirm all clauses evaluate true.

COINjecture generates 3-CNF instances near the phase transition threshold ($\alpha \approx 4.27$ clauses per variable), where instances are hardest. Quality is binary—partial satisfaction scores 0.0. Best known algorithms include CDCL (Conflict-Driven Clause Learning) and Schöning's randomised $O(2^{0.386n})$ for 3-SAT [16, 17].

d. Registry Parameters (Currently Implemented)

Problem	Class	Scaling Exp.	Verify Cost	Size Ratio	Max Size	Quality
Subset Sum	NP-C	0.80	$O(n)$	1.00	180	Binary
SAT (3-CNF)	NP-C	0.70	$O(n{\cdot}m)$	1.00	320	Binary
TSP	NP-H	0.50	$O(n)$	0.90	150	Gradient

NP-C = NP-Complete. NP-H = NP-Hard. Scale = scaling exponent. Size R. = size relative to Subset Sum at equivalent difficulty. Max = absolute safety ceiling, refined from observed testnet solve times.

e. Optimality (Current & Future Problems)

Decision problems (Subset Sum, SAT, Factorization) have binary quality: correct = 1.0, anything else = 0.0. There is no gradient—partial solutions receive zero work score.

Optimization problems (TSP, Graph Coloring, SVP) have continuous quality. Valid solutions always score above zero; better solutions score higher. Quality is scored relative to the best solution found in the current epoch, making it a competitive measure. Invalid solutions (missing cities, adjacent same-colour vertices, zero vector) score 0.0 regardless.

Problem	Optimal Solution	Quality Formula
Subset Sum	Any S' where $\sum s' = T$	1.0 if valid, else 0.0
SAT	Any satisfying assignment	1.0 if all clauses true, else 0.0
TSP	Shortest Hamiltonian tour	$1/(tour\_length + 1)$ vs. epoch-best
Graph Color.	Fewest colours, no conflicts	$1/colors\_used$ vs. epoch-best
Factorization	$p \times q = N$, non-trivial	1.0 if valid, else 0.0
SVP	Shortest non-zero lattice vector	$1/(\|v\| + 1)$ vs. epoch-best

f. Future Problem Types

Adding a new problem type requires implementing the `ProblemDescriptor` trait and calling `register()`. No changes are required to the difficulty adjuster, work score calculator, or consensus code. Other NP problems we intend to explore include:

Problem	Class	Scaling Exp.	Verify Cost	Size Ratio	Max Size	Quality
Graph Coloring	NP-C	0.65	$O(V+E)$	0.60	80	Gradient
Factorization	NP and co-NP	0.33	$O(M(\log N))$	0.50	200	Binary
SVP	NP-H	0.50	$O(n^2)$	0.40	40	Gradient

Difficulty Adjustment

The difficulty adjuster maintains stable block times by scaling problem size and is empirically tuned based on observed solve times. All targets are derived from network state and

mathematical constants—no arbitrary parameters.

a. **Target Times.** The optimal solve time is derived from the network's own median block time: $optimal = median\_block\_time \times \eta$, where $\eta = 1/\sqrt{2}$. The acceptable range is bounded by the golden ratio: $min\_target = optimal \times \varphi^{-1}$, $max\_target = optimal \times \varphi$. During bootstrap (blocks 0–19), seeded defaults govern; after block 20, empirical data takes over.

b. **Adjustment Formula.** Given the moving average of the last N block solve times, the adjuster computes a dimensionless time ratio: $time\_ratio = avg\_solve\_time / optimal$. Problem size scales as: $new\_size = current\_size \times (1/time\_ratio)^{0.5}$, clamped to [0.4, 2.0] to prevent extreme jumps. The square root provides smooth convergence without oscillation.

c. **High Variance Deferral.** If standard deviation exceeds 80% of the mean ($\sigma > 0.8\mu$), the adjuster defers—it does not adjust difficulty until the window stabilises. This prevents overreaction to transient spikes.

d. **Stall Recovery.** If average solve time exceeds $2 \times max\_target$, the adjuster applies a stall penalty ($size \times 0.7$) and enters recovery mode, gradually restoring size in increments of 1 once $time\_ratio$ falls below 1.2. Mining failures trigger a softer penalty ($size \times 0.85$). These values have been empirically tuned during testnet operation.

User Submissions Integration

The network maintains two problem sources: (1) consensus problems generated deterministically from network state for base rewards, and (2) user-submitted problems with escrowed bounties. User submissions create a computational marketplace where the mining network solves real problems for payment.

Every submission requires an escrowed bounty, a minimum work score threshold, and an expiration window. The bounty locks at submission and resolves in exactly one of three ways: released to a solver on valid solution, refunded to the submitter on cancellation (open problems only), or refunded on expiration. No partial claims. No intermediate access. Bounties finalise at confirmation depth, and depth scales with bounty value (W-bits).

Users may select one of two submission modes.

a. Public Submissions

Full problem specification broadcast at submission. Miners inspect, evaluate, and commit. Appropriate when the problem carries no proprietary value.

b. Private Submissions

Submitter publishes a cryptographic commitment, a well-formedness proof, and public parameters including a complexity estimate. On the current testnet, this proof is a SHA-256 MAC placeholder. A reveal phase is required before solutions are accepted—the submitter opens the commitment, the network verifies the match, and solving proceeds as in public mode. A production zero-knowledge (ZK) proof will ensure that the committed instance is

valid and at the claimed difficulty is accurate without revealing it. A formalized ZK proof is future work and is not yet deployed.

c. Aggregation Strategies

Strategy	Resolution	Use Case
Any	First valid solution	Decision problems (Subset Sum, SAT)
Best	Highest quality in competition window	Optimisation problems (TSP, Graph Coloring)
Multiple	N distinct valid solutions, bounty split	Diverse solution sets
Statistical	K solutions, aggregate statistics, bounty split	Research and distribution analysis

Verify Lean Proofs Independently

```
git clone https://github.com/COINjecture-Network/COINjecture2.0.git
cd COINjecture2.0/lean4
lake exe cache get
lake build
```

The build succeeds with zero errors on Lean 4 v4.28.0 with Mathlib v4.28.0. The complete source is available at:
https://github.com/COINjecture-Network/COINjecture2.0/tree/main/lean4